



CodeSmile

Test Plan

Pre – modification

RIFERIMENTO	TestPlan-PreModification
VERSIONE	0.6
DESTINATARIO	Prof. Andrea De Lucia Dott. Giammaria Giordano



Laurea Magistrale in Software Engineering & IT Management –Università di Salerno
Corso di Ingegneria del Software: tecniche avanzate
Prof. Andrea De Lucia, Dott. Giammaria Giordano

Gestione Gruppo

MEMBRO	EMAIL	MATRICOLA
MARTINA DE LUCIA	m.delucia18@studenti.unisa.it	NF22500053
ALESSANDRA RAIA	a.raia7@studenti.unisa.it	NF22500054
CARLA STEFANILE	c.stefanile1@studenti.unisa.it	NF22500097



Version

DATA	VERSIONE	DESCRIZIONE	AUTORI
21/01/2026	0.1	Prima Stesura	MDL, AR, CS
21/01/2026	0.2	Seconda Stesura	AR
21/01/2026	0.3	Completata la Stesura	AR
21/01/2026	0.4	Prima Revisione	MDL
21/01/2026	0.5	Seconda Revisione	CS
15/02/2026	0.6	Ultima Revisione	MDL



Sommario

Gestione Gruppo.....	2
Version.....	3
1 Introduzione.....	5
2 Riferimenti ad altri documenti.....	5
3 Ambiente di Testing.....	5
4 Approccio.....	6
4.1 Esecuzione dei Test di Unità e di Integrazione esistenti.....	6
4.2 Progettazione del testing di sistema tramite Category Partition.....	6
4.2.1 Nota metodologica.....	6
5 Criteri Passed/Failed	7
6 Sospensione e ripristino.....	7



1 Introduzione

Il documento di Test Plan ha l'obiettivo di definire le strategie che verranno utilizzate per effettuare il testing del progetto CodeSmile, **prima** dell'implementazione delle modifiche previste dalle Change Request.

Lo scopo principale è verificare il corretto funzionamento dei moduli esistenti del sistema, con particolare attenzione al flusso di utilizzo tramite **GUI**, al fine di stabilire una **baseline di riferimento** solida e verificabile.

Tale baseline sarà utilizzata per confrontare i risultati dei test successivi all'introduzione delle funzionalità additive previste dalla CR01.

Il documento si colloca quindi in una fase *pre-modification* del ciclo di vita del software e non include attività di testing specifiche relative alle nuove funzionalità LLM-based, che saranno oggetto di un test plan separato.

2 Riferimenti ad altri documenti

- **Change Request [ChangeRequest]**
Documento di riferimento che descrive le funzionalità additive proposte, le componenti coinvolte e i requisiti funzionali e non funzionali associati.
- **Use Case Model [UseCase]**
Documento separato contenente la modellazione dei casi d'uso del sistema, utile come supporto concettuale per il testing di sistema.
- **Test Case Specification Pre Modification [TestCaseSpecification-PreModification]**
Documento che esplicita il Category Partition sul sistema prima di effettuare le modifiche.
- **Test Incident Report Pre Modification [TestIncidentReport-PreModification]**
Documento che colleziona i risultati dei test effettuati.

3 Ambiente di Testing

- Sistema Operativo: Windows 11
- Python: 3.11
- Dipendenze: installate come previsto attraverso il file *requirements.txt*
- Repository: https://github.com/alessraia/smell_ai

L'ambiente di test riproduce fedelmente una configurazione di utilizzo reale del sistema, coerente con l'esecuzione tramite interfaccia grafica.



4 Approccio

L'approccio di testing adottato è di tipo **incrementale e baseline-oriented**, ed è finalizzato a valutare lo stato attuale del sistema prima dell'introduzione delle modifiche previste dalle Change Request.

Poiché CodeSmile è un sistema già esistente e dispone di una suite di test automatici, l'attività di testing pre-modification è stata strutturata in modo da:

- verificare la stabilità dei componenti e delle loro interazioni,
- progettare in modo esplicito e documentato il testing di sistema rilevante per il flusso GUI.

L'approccio seguito si articola nelle fasi descritte di seguito.

4.1 Esecuzione dei Test di Unità e di Integrazione esistenti

In una prima fase vengono eseguiti tutti i **test di unità** e **test di integrazione** già presenti nel progetto.

L'obiettivo di questa attività è:

- verificare il corretto funzionamento dei singoli componenti software;
- validare le principali interazioni tra moduli, servizi e pipeline di analisi;
- accertare che il sistema si trovi in uno stato stabile prima di qualsiasi modifica.

I risultati di questa fase costituiscono la baseline tecnica del sistema a livello di componenti e integrazione.

4.2 Progettazione del testing di sistema tramite Category Partition

Al fine di ottenere una baseline di testing di sistema solida, ripetibile e confrontabile nel tempo, si è scelto di **progettare ex novo** il testing di sistema utilizzando la tecnica di **Category Partition**.

La progettazione dei casi di test è basata su:

- i flussi di utilizzo della GUI,
- i casi d'uso principali e alternativi del sistema,
- le modalità di selezione degli input e degli output,
- le condizioni di errore e i casi limite legati all'interazione utente-sistema.

La definizione delle categorie, delle scelte e dei vincoli è raccolta in un documento di *Test Case Specification Pre Modification* separato, al fine di mantenere il presente Test Plan focalizzato sulla strategia di testing.

4.2.1 Nota metodologica

La scelta di progettare da zero il testing di sistema tramite Category Partition è motivata dalla necessità di disporre di una suite di test:

- formalmente progettata,
- esplicitamente documentata,



- facilmente confrontabile prima e dopo l'introduzione della CR01.

Tale scelta non invalida i test esistenti, ma ne integra i risultati all'interno di un processo di testing più strutturato e controllabile.

L'attività di creazione ed esecuzione dei test è stata fortemente supportata da una preliminare esecuzione manuale, necessaria prima di adottare l'automazione dei test, per validare la coerenza della test suite con il comportamento reale, osservato, del sistema già esistente e prodotto da un team diverso.

5 Criteri Passed/Failed

Un test è considerato **superato (PASSED)** se il comportamento osservato del sistema è conforme al comportamento atteso definito nel caso di test, senza generare errori non gestiti, crash o risultati incoerenti.

Un test è considerato **fallito (FAILED)** se si verifica almeno una delle seguenti condizioni:

- il sistema produce un output diverso da quello atteso;
- la GUI non consente di completare correttamente il flusso previsto;
- si verificano eccezioni non gestite o arresti anomali;
- l'analisi non viene completata a fronte di input validi.

L'attività di testing pre-modification è considerata **completata con successo** se:

- tutti i test di unità e di integrazione risultano superati;
- i test di sistema progettati tramite Category Partition non evidenziano difetti bloccanti;
- eventuali anomalie riscontrate sono documentate e non compromettono il flusso standard di utilizzo della GUI.

6 Sospensione e ripristino

In questa sezione verranno specificati i criteri di sospensione del test e di ripristino.

Criteri di sospensione

Il testing non verrà sospeso fino alla sua terminazione, anche in caso di rilevazione di una failure. Il testing potrà essere momentaneamente sospeso nel caso venga restituito, al momento dell'esecuzione, un errore nella definizione di uno dei test stessi.

Criterio di ripristino

Il testing verrà ripreso dopo aver risolto i fault individuati